

Database System Engineering and Distributed Backend Development

Course Code: 25CS1302E

Week-2

Step 1 — Create Database

```
MySQL 8.0 Command Line Client
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> DROP DATABASE IF EXISTS bookflow_db;
Query OK, 2 rows affected (0.09 sec)

mysql> CREATE DATABASE bookflow_db;
Query OK, 1 row affected (0.05 sec)

mysql> USE bookflow_db;
Database changed
mysql>
```

Step 2 — Create Books Table (DDL + Constraints)

```
CREATE TABLE Books (
  book_id INT PRIMARY KEY,
  title VARCHAR(100) NOT NULL,
  isbn VARCHAR(20) UNIQUE,
  published_year INT CHECK (published_year < 2027)
);
```

```
MySQL 8.0 Command Line Client
Database changed
mysql> CREATE TABLE Books (
->   book_id INT PRIMARY KEY,
->   title VARCHAR(100) NOT NULL,
->   isbn VARCHAR(20) UNIQUE,
->   published_year INT CHECK (published_year < 2027)
-> );
Query OK, 0 rows affected (0.13 sec)

mysql> INSERT INTO Books (book_id, title, isbn, published_year)
-> VALUES
-> (1, 'The Great Gatsby', '9780743273565', 1925),
-> (2, 'To Kill a Mockingbird', '9780061120084', 1960),
-> (3, '1984', '9780451524935', 1949);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
```

Step 3 & 4 — Insert Books and Display

```
INSERT INTO Books (book_id, title, isbn, published_year) VALUES
(1, 'The Great Gatsby', '9780743273565', 1925),
(2, 'To Kill a Mockingbird', '9780061120084', 1960),
(3, '1984', '9780451524935', 1949);

SELECT * FROM Books;
```

```
MySQL 8.0 Command Line Client
->   book_id INT PRIMARY KEY,
->   title VARCHAR(100) NOT NULL,
->   isbn VARCHAR(20) UNIQUE,
->   published_year INT CHECK (published_year < 2027)
-> );
Query OK, 0 rows affected (0.13 sec)

mysql> INSERT INTO Books (book_id, title, isbn, published_year)
-> VALUES
-> (1, 'The Great Gatsby', '9780743273565', 1925),
-> (2, 'To Kill a Mockingbird', '9780061120084', 1960),
-> (3, '1984', '9780451524935', 1949);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Books;
+-----+-----+-----+-----+
| book_id | title                | isbn                | published_year |
+-----+-----+-----+-----+
| 1       | The Great Gatsby     | 9780743273565      | 1925           |
| 2       | To Kill a Mockingbird | 9780061120084      | 1960           |
| 3       | 1984                 | 9780451524935      | 1949           |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql>
```

Step 5, 6 & 7 — Members Table: Create, Insert, Display

```

CREATE TABLE Members (
  member_id INT PRIMARY KEY,
  full_name VARCHAR(100),
  email VARCHAR(100) UNIQUE
);

INSERT INTO Members (member_id, full_name, email) VALUES
(101, 'John Smith', 'john.smith@email.com'),
(102, 'Emma Wilson', 'emma.wilson@email.com'),
(103, 'Michael Brown', 'michael.brown@email.com');

SELECT * FROM Members;

```

```

mysql> CREATE TABLE Members (
->   member_id INT PRIMARY KEY,
->   full_name VARCHAR(100),
->   email VARCHAR(100) UNIQUE
-> );
Query OK, 0 rows affected (0.14 sec)

mysql> INSERT INTO Members (member_id, full_name, email)
-> VALUES
-> (101, 'John Smith', 'john.smith@email.com'),
-> (102, 'Emma Wilson', 'emma.wilson@email.com'),
-> (103, 'Michael Brown', 'michael.brown@email.com');
Query OK, 3 rows affected (0.06 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Members;
+-----+-----+-----+
| member_id | full_name | email |
+-----+-----+-----+
| 101 | John Smith | john.smith@email.com |
| 102 | Emma Wilson | emma.wilson@email.com |
| 103 | Michael Brown | michael.brown@email.com |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql>

```

Step 8 — Create Loans Table (FOREIGN KEYS)

```

CREATE TABLE Loans (
  loan_id INT PRIMARY KEY,
  member_id INT,
  book_id INT,
  loan_date DATE,
  FOREIGN KEY (member_id) REFERENCES Members(member_id),
  FOREIGN KEY (book_id) REFERENCES Books(book_id)
);

```

Step 9 & 10 — Insert 10 Loans and Display

```

INSERT INTO Loans (loan_id, member_id, book_id, loan_date) VALUES
(1, 101, 1, '2025-01-05'), (2, 102, 2, '2025-01-08'),
(3, 103, 3, '2025-01-10'), (4, 101, 2, '2025-02-01'),
(5, 102, 1, '2025-02-05'), (6, 103, 2, '2025-02-12'),
(7, 101, 3, '2025-03-01'), (8, 102, 3, '2025-03-07'),
(9, 103, 1, '2025-03-15'), (10, 101, 1, '2025-04-01');

SELECT * FROM Loans;

```

```
MySQL 8.0 Command Line Client

mysql> CREATE TABLE Loans (
->   loan_id INT PRIMARY KEY,
->   member_id INT,
->   book_id INT,
->   loan_date DATE,
->   FOREIGN KEY (member_id) REFERENCES Members(member_id),
->   FOREIGN KEY (book_id) REFERENCES Books(book_id)
-> );
Query OK, 0 rows affected (0.17 sec)

mysql> INSERT INTO Loans (loan_id, member_id, book_id, loan_date)
-> VALUES
-> (1,101,1,'2025-01-05'),
-> (2,102,2,'2025-01-08'),
-> (3,103,3,'2025-01-10'),
-> (4,101,2,'2025-02-01'),
-> (5,102,1,'2025-02-05'),
-> (6,103,2,'2025-02-12'),
-> (7,101,3,'2025-03-01'),
-> (8,102,3,'2025-03-07'),
-> (9,103,1,'2025-03-15'),
-> (10,101,1,'2025-04-01');
Query OK, 10 rows affected (0.06 sec)
Records: 10  Duplicates: 0  Warnings: 0

mysql>
```

```
MySQL 8.0 Command Line Client

-> (5,102,1,'2025-02-05'),
-> (6,103,2,'2025-02-12'),
-> (7,101,3,'2025-03-01'),
-> (8,102,3,'2025-03-07'),
-> (9,103,1,'2025-03-15'),
-> (10,101,1,'2025-04-01');
Query OK, 10 rows affected (0.06 sec)
Records: 10  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Loans;
+-----+-----+-----+-----+
| loan_id | member_id | book_id | loan_date |
+-----+-----+-----+-----+
| 1 | 101 | 1 | 2025-01-05 |
| 2 | 102 | 2 | 2025-01-08 |
| 3 | 103 | 3 | 2025-01-10 |
| 4 | 101 | 2 | 2025-02-01 |
| 5 | 102 | 1 | 2025-02-05 |
| 6 | 103 | 2 | 2025-02-12 |
| 7 | 101 | 3 | 2025-03-01 |
| 8 | 102 | 3 | 2025-03-07 |
| 9 | 103 | 1 | 2025-03-15 |
| 10 | 101 | 1 | 2025-04-01 |
+-----+-----+-----+-----+
10 rows in set (0.00 sec)

mysql>
```

Step 11 — JOIN Query (Who Borrowed What)

```
SELECT m.full_name AS Member_Name, b.title AS Book_Title
FROM Loans l
INNER JOIN Members m ON l.member_id = m.member_id
INNER JOIN Books b ON l.book_id = b.book_id;
```

```
MySQL 8.0 Command Line Client
10 rows in set (0.00 sec)

mysql> SELECT m.full_name AS Member_Name,
->      b.title AS Book_Title
-> FROM Loans l
-> INNER JOIN Members m
-> ON l.member_id = m.member_id
-> INNER JOIN Books b
-> ON l.book_id = b.book_id;
+-----+-----+
| Member_Name | Book_Title |
+-----+-----+
| John Smith  | The Great Gatsby |
| John Smith  | To Kill a Mockingbird |
| John Smith  | 1984 |
| John Smith  | The Great Gatsby |
| Emma Wilson | To Kill a Mockingbird |
| Emma Wilson | The Great Gatsby |
| Emma Wilson | 1984 |
| Michael Brown | 1984 |
| Michael Brown | To Kill a Mockingbird |
| Michael Brown | The Great Gatsby |
+-----+-----+
10 rows in set (0.00 sec)

mysql> _
```

Step 12 — GROUP BY Query (Books per Year)

```
SELECT published_year, COUNT(book_id) AS Total_Books
FROM Books
GROUP BY published_year
ORDER BY published_year;
```

```
MySQL 8.0 Command Line Client
+-----+-----+
| Emma Wilson | The Great Gatsby |
| Emma Wilson | 1984 |
| Michael Brown | 1984 |
| Michael Brown | To Kill a Mockingbird |
| Michael Brown | The Great Gatsby |
+-----+-----+
10 rows in set (0.00 sec)

mysql> SELECT published_year,
->      COUNT(book_id) AS Total_Books
-> FROM Books
-> GROUP BY published_year
-> ORDER BY published_year;
+-----+-----+
| published_year | Total_Books |
+-----+-----+
| 1925 | 1 |
| 1949 | 1 |
| 1960 | 1 |
+-----+-----+
3 rows in set (0.04 sec)

mysql> _
```

Step 13-16 — Donation via TRANSACTION (Atomicity)

```

CREATE TABLE Donation_History (
  donation_id INT PRIMARY KEY,
  book_id INT,
  donor_name VARCHAR(100),
  donation_date DATE,
  FOREIGN KEY (book_id) REFERENCES Books(book_id)
);

START TRANSACTION;

INSERT INTO Books (book_id, title, isbn, published_year)
VALUES (4, 'Animal Farm', '9780451526342', 1945);

INSERT INTO Donation_History (donation_id, book_id, donor_name, donation_date)
VALUES (1, 4, 'Raj Kumar', CURDATE());

COMMIT;

```

```

MySQL 8.0 Command Line Client
+-----+-----+
| published_year | Total_Books |
+-----+-----+
| 1925 | 1 |
| 1949 | 1 |
| 1960 | 1 |
+-----+-----+
3 rows in set (0.04 sec)

mysql> CREATE TABLE Donation_History (
  -> donation_id INT PRIMARY KEY,
  -> book_id INT,
  -> donor_name VARCHAR(100),
  -> donation_date DATE,
  -> FOREIGN KEY (book_id)
  -> REFERENCES Books(book_id)
  -> );
Query OK, 0 rows affected (0.07 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> _

```

Step 17 & 18 — Index on ISBN + Fast Search

```

CREATE INDEX idx_books_isbn ON Books(isbn);

SELECT * FROM Books WHERE isbn = '9780451524935';

```

```
MySQL 8.0 Command Line Client

+----+-----+-----+-----+
| book_id | title | isbn | published_year |
+----+-----+-----+-----+
| 1 | The Great Gatsby | 9780743273565 | 1925 |
| 2 | To Kill a Mockingbird | 9780061120084 | 1960 |
| 3 | 1984 | 9780451524935 | 1949 |
| 4 | Animal Farm | 9780451526342 | 1945 |
+----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> CREATE INDEX idx_books_isbn
-> ON Books(isbn);
Query OK, 0 rows affected (0.11 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> SELECT *
-> FROM Books
-> WHERE isbn='9780451524935';
+----+-----+-----+-----+
| book_id | title | isbn | published_year |
+----+-----+-----+-----+
| 3 | 1984 | 9780451524935 | 1949 |
+----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> _
```

Final State of All Tables

```
MySQL 8.0 Command Line Client

Query OK, 1 row affected (0.04 sec)

mysql> COMMIT;
Query OK, 0 rows affected (0.05 sec)

mysql> SELECT * FROM Donation_History;
+----+-----+-----+-----+
| donation_id | book_id | donor_name | donation_date |
+----+-----+-----+-----+
| 1 | 4 | Raj Kumar | 2026-07-30 |
+----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT * FROM Books;
+----+-----+-----+-----+
| book_id | title | isbn | published_year |
+----+-----+-----+-----+
| 1 | The Great Gatsby | 9780743273565 | 1925 |
| 2 | To Kill a Mockingbird | 9780061120084 | 1960 |
| 3 | 1984 | 9780451524935 | 1949 |
| 4 | Animal Farm | 9780451526342 | 1945 |
+----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> _
```



```
MySQL 8.0 Command Line Client
+----+-----+-----+-----+
| book_id | title | isbn | published_year |
+----+-----+-----+-----+
| 1 | The Great Gatsby | 9780743273565 | 1925 |
| 2 | To Kill a Mockingbird | 9780061120084 | 1960 |
| 3 | 1984 | 9780451524935 | 1949 |
| 4 | Animal Farm | 9780451526342 | 1945 |
+----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> CREATE INDEX idx_books_isbn
-> ON Books(isbn);
Query OK, 0 rows affected (0.11 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> SELECT *
-> FROM Books
-> WHERE isbn='9780451524935';
+----+-----+-----+-----+
| book_id | title | isbn | published_year |
+----+-----+-----+-----+
| 3 | 1984 | 9780451524935 | 1949 |
+----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> _
```

Week 2 Viva Questions & Answers

1. What is the purpose of the Loans table in the BookFlow database?

Answer:

The **Loans** table acts as a **junction (bridge) table** between the **Members** and **Books** tables. It stores borrowing transactions and helps track **which member has borrowed which book and on what date**.

2. Why do we use JOIN operations in SQL?

Answer:

JOIN is used to combine data from two or more related tables based on a common column. In BookFlow, JOIN is used to display **member names** and **book titles** together by connecting the **Members**, **Loans**, and **Books** tables.

3. What is the purpose of the GROUP BY clause?

Answer:

The **GROUP BY** clause groups rows that have the same values in a specified column. It is commonly used with aggregate functions such as **COUNT()**, **SUM()**, and **AVG()**. In this project, it is used to **count the number of books published in each year**.

4. What is a Transaction, and why is it important?

Answer:

A **Transaction** is a sequence of SQL operations treated as a single unit of work. It ensures that **all operations either complete successfully (COMMIT) or none are applied (ROLLBACK)** if an error occurs. This maintains **data consistency and integrity**.

5. Why is an Index created on the ISBN column?

Answer:

An **Index** is created on the **ISBN** column to improve search performance. Since ISBN values are unique and frequently searched, the database can locate records much faster using the index instead of scanning every row in the **Books** table. This reduces query execution time and improves efficiency.

.